FORMATION

Analyse de données avec Python

Jour 1, après-midi
Sélection, transformation et nettoyage de données

Cours et TP2 sur le jeu de données des ventes

Le déroulé de l'après-midi

1	2	3	4		5	6	7	8
Sélection	**Transformer**	**Texte**	**Types**	**Pause**	**Manquants et doublons**	**Tri et index**	**Démarche**	**Parquet**
TP2 partie 1	TP2 partie 2	TP2 partie 3	TP2 partie 4		TP2 partie 5	TP2 partie 6	TP2 partie 7	TP2 partie 8

13h30

début

15h00

pause

17h00

fin

Chaque section de cours est suivie de la partie du TP2 qui lui correspond.

D'où l'on part : les constats du TP1

Constat	Dans le fichier des ventes
Valeurs manquantes	3 régions vides
Valeurs aberrantes	une quantité à -3, un prix à 0, un prix à 12 999, une commande de 480 unités
Variantes d'écriture	`Bretagne ` avec un espace, `occitanie`, `Ile-de-France`, `INFORMATIQUE`
Doublons	2 lignes identiques
Types	`date` lue comme du texte
Noms de colonnes	`Prix unitaire` et `Client fidèle` au milieu de noms en minuscules

01

Sélectionner des données

loc et iloc | Filtrage booléen | Modifier une sélection

1 loc et iloc

	loc	iloc
Sélectionne par	label	position
Fin de tranche	incluse	exclue
Masque booléen	oui	non

La syntaxe est toujours `df.loc[lignes, colonnes]`. Les deux-points seuls signifient « tout ».

```
df.loc[0]                                  # label 0
df.loc[0:4, ["produit", "quantite"]]   # 5 lignes
df.iloc[0:4, [4, 6]]                       # 4 lignes

df.iloc[-1]          # dernière ligne
df.iloc[:, 0]        # première colonne
df.loc[:, "produit"]
```

1 Label ou position : la différence après un tri

```
tri = df.sort_values("quantite", ascending=False)

tri.loc[0]     # la ligne de label 0,
               # où qu'elle soit après le tri

tri.iloc[0]    # la nouvelle première ligne :
               # la plus grosse quantité
```

- Avec l'index par défaut, labels et positions coïncident : la différence ne se voit pas

Elle apparaît dès qu'on **trie**, **filtre** ou **change d'index**
`loc` suit les lignes, `iloc` suit l'ordre d'affichage

1 Le filtrage booléen

```
df["quantite"] > 10          # masque True / False
df[df["quantite"] > 10]      # lignes filtrées

df[(df["categorie"] == "Bureau")
   & (df["quantite"] >= 5)]

df[(df["quantite"] <= 0)
   | (df["quantite"] > 100)]

df[~df["Client fidèle"]]
```

Opérateur	Sens
`&`	et
`|`	ou
`~`	non

Chaque condition entre parenthèses.

1 Les deux erreurs classiques

Ne fonctionne pas

```
df[df["quantite"] > 5 and df["quantite"] <
10]
# ValueError: The truth value of a
# Series is ambiguous

df[df["quantite"] > 5 & df["quantite"] < 10]
# erreur : & est prioritaire sur > et <
```

Fonctionne

```
df[(df["quantite"] > 5) & (df["quantite"] < 10)]

# ou, plus lisible ici :
df[df["quantite"].between(6, 9)]
```

`and` et `or` comparent deux valeurs uniques. Pour comparer deux colonnes ligne par ligne, il faut `&` et `|`.

1 Construire des conditions

Méthode	Exemple	Garde les lignes où...
`isin`	`df["region"].isin(["Bretagne", "Occitanie"])`	la valeur est dans la liste
`between`	`df["Prix unitaire"].between(10, 50)`	la valeur est dans l'intervalle, bornes incluses
`isna`	`df["region"].isna()`	la valeur est manquante
`str.contains`	`df["produit"].str.contains("Écran")`	le texte contient le pattern

Sélection combinée, et query pour les filtres longs

```
df.loc[df["quantite"] > 10, ["date", "produit", "quantite"]]

df.query("quantite > 10 and categorie == 'Bureau'")
df.query("`Prix unitaire` > 5000")     # espaces : accents graves
```

1 Modifier une sélection

Sans effet

```
df[df["quantite"] > 100]["quantite"] = 0
# ChainedAssignmentError
```

Deux étapes : la sélection crée une copie temporaire, c'est elle qui est modifiée, puis elle est perdue.

Correct

```
df.loc[df["quantite"] > 100, "quantite"] = 0

# sous-ensemble à modifier ensuite
bureau = df[df["categorie"] == "Bureau"].copy()
```

Une seule étape avec `loc` : sélection et affectation portent sur `df` lui-même.

Depuis pandas 3, le mode Copy-on-Write est actif par défaut : toute sélection se comporte comme une copie. Avec pandas 2, le même code affichait un `SettingWithCopyWarning`.

1 Pratique : TP2, partie 1

Sélectionner des données, dans le notebook `exercices/tp02-nettoyage/tp02.ipynb`

Questions	Ce qu'on pratique
1.1	les mêmes lignes avec `loc` puis avec `iloc` : pourquoi les bornes diffèrent
1.2 et 1.3	combiner des conditions, `isin`, et ce que le filtre laisse échapper
1.4 à 1.6	`loc` avec un masque et des colonnes, `~` et `between`, `query`
1.7	modifier une copie en une seule instruction avec `loc`

Comme au TP1 : **résultat attendu**, cellules de **vérification** avec `assert`, questions **Pour aller plus loin** facultatives.

02

Transformer les données

Créer et supprimer | map et apply | pd.cut

2 Créer une colonne

```
df["montant"] = df["Prix unitaire"] * df["quantite"]
df["montant_ttc"] = (df["montant"] * 1.2).round(2)
df["source"] = "boutique"

df["taille_commande"] = np.where(
    df["quantite"] >= 10, "grosse", "standard"
)
```

- On affecte un calcul vectorisé à un nouveau nom de colonne
- Une valeur seule est répétée sur toutes les lignes

`np.where(condition, si_vrai, si_faux)` : le `if ... else` vectorisé

2 assign : enrichir sans modifier l'original

Avec assign

```
df_enrichi = df.assign(
    montant=df["Prix unitaire"] * df["quantite"],
    annee=df["date"].dt.year,
)

"montant" in df.columns           # False
"montant" in df_enrichi.columns   # True
```

`assign` renvoie un **nouveau** DataFrame avec les colonnes ajoutées. `df` reste tel qu'il était.

L'équivalent sans assign

```
df_enrichi = df.copy()
df_enrichi["montant"] = (
    df_enrichi["Prix unitaire"] *
df_enrichi["quantite"]
)

df_enrichi["annee"] = df_enrichi["date"].dt.year
```

On copie d'abord, puis on ajoute les colonnes sur la copie. Sans `copy()`, ces affectations écriraient dans `df`.

2 Supprimer des colonnes ou des lignes

```
df = df.drop(columns=["source", "montant_ttc"])

df = df.drop(index=[0, 1])    # par label
```

`drop` renvoie un **nouveau** DataFrame : il faut réaffecter le résultat

`inplace=True` existe encore, mais il est déconseillé

- Il n'apporte aucun gain et empêche d'enchaîner les opérations

2 map : remplacer valeur par valeur

```
codes = {
    "Bureau": "BUR",
    "Papeterie": "PAP",
    "Informatique": "INF",
    "Électroménager": "ELM",
}
df["code_categorie"] = df["categorie"].map(codes)

# avec une fonction
df["premier_mot"] = df["produit"].map(
    lambda p: p.split()[0]
)
```

S'applique à une **Series**

- Avec un dictionnaire : chaque valeur est remplacée par sa correspondance

Les valeurs absentes du dictionnaire deviennent `NaN`

Ici, `INFORMATIQUE` en majuscules donne `NaN` : un bon moyen de repérer les valeurs inattendues, à condition de vérifier le résultat.

2 apply : une fonction par ligne

```
def libelle(ligne):
    return f"{ligne['produit']} x{ligne['quantite']}"

df["libelle"] = df.apply(libelle, axis=1)
# "Lampe LED x9"
```

Sur une Series : une fonction par valeur, comme `map`
Sur un DataFrame avec `axis=1` : une fonction par **ligne**

- Pratique quand le calcul combine plusieurs colonnes

Mais c'est une boucle Python : **lent** sur de gros volumes

2 Du plus rapide au plus lent

1

Opérations vectorisées

Opérateurs, méthodes pandas, `np.where`, accesseurs `.str` et `.dt`.

2

map avec un dictionnaire

Pour remplacer des valeurs selon une table de correspondance.

3

map ou apply avec une fonction

En dernier recours, quand rien d'autre ne convient.

Le libellé précédent, sans apply

```
df["libelle"] = df["produit"] + " x" + df["quantite"].astype(str)
```

2 Découper en tranches avec pd.cut

```
df["gamme"] = pd.cut(
    df["Prix unitaire"],
    bins=[0, 30, 100, 500],
    labels=["petit prix", "prix moyen", "haut de gamme"],
)
```

`np.where` choisit entre deux valeurs. `pd.cut` range une valeur numérique dans plus de deux classes, selon des bornes.

Prix unitaire	gamme
30	petit prix
30,01	prix moyen
189	haut de gamme
0 ou 12 999	`NaN`

Borne de droite incluse, borne de gauche exclue. Le résultat est de type `category`.

2 Pratique : TP2, partie 2

Transformer les données, sur un petit tableau de commandes propre

Questions	Ce qu'on pratique
2.1	créer des colonnes par un calcul vectorisé et avec `np.where`
2.2	`assign` sans modifier l'original, puis `drop`
2.3	`map` avec un dictionnaire, puis avec une `lambda`
2.4	un libellé avec `apply`, puis le même sans `apply`

03

Nettoyer le texte

Espaces et casse | Remplacer | Chercher et extraire

3 L'accesseur .str

Méthode	Rôle
`.str.strip()`	retirer les espaces aux extrémités
`.str.lower()`, `.str.upper()`	changer la casse
`.str.replace(a, b)`	remplacer un pattern
`.str.contains(m)`	tester la présence d'un pattern
`.str.len()`	longueur
`.str.split(sep)`	découper
`.str.extract(regex)`	extraire une partie

```
df["region"].str.strip()
df["categorie"].str.lower()

# les méthodes s'enchaînent
df["region"].str.strip().str.lower()
```

- Les méthodes de texte de Python, appliquées à toute une colonne
- Les valeurs manquantes restent manquantes, sans erreur

3 Corriger des variantes d'écriture

Le piège de title()

```
"Île-de-France".title()
# 'Île-De-France'

# une majuscule après chaque tiret
```

Pour des libellés de référence, on corrige explicitement les variantes plutôt que de compter sur une règle générale.

Un dictionnaire de corrections

```
corrections = {
    "occitanie": "Occitanie",
    "Ile-de-France": "Île-de-France",
}

df["region"] = (
    df["region"]
    .str.strip()
    .replace(corrections)
)
```

3 replace ou str.replace ?

`Series.replace` : valeurs entières

```
df["region"].replace({"occitanie": "Occitanie"})

# remplace uniquement les cellules
# qui valent exactement "occitanie"
```

`Series.str.replace` : morceaux de texte

```
df["produit"].str.replace(" ", "_")
# "Lampe LED" devient "Lampe_LED"

# expression régulière
df["produit"].str.replace(r"\s+", " ", regex=True)
```

Par défaut, `.str.replace` cherche un texte littéral. Avec `regex=True`, le pattern est une expression régulière.

3 Chercher et extraire

```
df["produit"].str.contains("écran", case=False)

df["pouces"] = df["produit"].str.extract(r"(\d+) pouces")
# "Écran 24 pouces" donne "24", les autres NaN

df[["type", "detail"]] = df["produit"].str.split(
    " ", n=1, expand=True
)
# "Lampe LED" donne "Lampe" et "LED"
```

`extract` récupère la partie entre parenthèses

- Par défaut, il renvoie un DataFrame : une colonne par partie entre parenthèses

Une seule partie et `expand=False` : une Series, qu'on peut convertir

```
references = pd.Series(
    ["REF-001 / Clavier", "REF-042 / Souris"])
references.str.extract(
    r"REF-(\d+)", expand=False).astype(int)
# 1, 42
```

3 Quelques patterns d'expressions régulières

Pattern	Signification
`\d`	un chiffre
`\D`	tout caractère qui n'est pas un chiffre
`\s`	un espace
`+`	une ou plusieurs fois
`^`, `$`	début, fin de la chaîne
`[A-Z]`	une lettre majuscule
`( )`	la partie à extraire

Ne garder que les chiffres

```
telephones = pd.Series(["01 23 45 67 89",
                        "01.23.45.67.89",
                        "01-23-45-67-89"])

telephones.str.replace(r"\D", "", regex=True)
# "0123456789" pour les trois
```

`\D` désigne tout ce qui n'est pas un chiffre : espaces, points et tirets disparaissent.

3 Pratique : TP2, partie 3

Nettoyer le texte avec l'accesseur `.str`

Questions	Ce qu'on pratique
3.1	nettoyer des noms de villes : espaces, majuscules, valeur manquante
3.2	compter les produits qui contiennent un mot, sans tenir compte de la casse
3.3	extraire un numéro avec `str.extract`, découper avec `str.split`

04

Gérer les types

astype | to_numeric et to_datetime | category

4 Pourquoi les types comptent

1

Les calculs

Des nombres stockés en texte ne s'additionnent pas : `"3" + "2"` donne `"32"`.

2

Les tris

Des dates en texte au format jour/mois/année se trient mal : `"02/01/2026"` passe avant `"15/12/2025"`.

3

La mémoire

Une colonne texte qui répète quelques valeurs occupe bien plus de place qu'une `category`.

Conversions simples

```
df["quantite"].astype("float64")
df["id_commande"].astype(str)

pd.Series([1, None, 3], dtype="Int64")   # entier avec valeur manquante : 1, <NA>, 3
```

4 Convertir sans planter : errors="coerce"

Nombres

```
prix = pd.Series(["12.5", "12,5", "abc"])

pd.to_numeric(prix, errors="coerce")
# 12.5, NaN, NaN

pd.to_numeric(prix.str.replace(",", "."),
              errors="coerce")
# 12.5, 12.5, NaN
```

Dates

```
df["date"] = pd.to_datetime(df["date"],

format="%Y-%m-%d")

pd.to_datetime(pd.Series(["03/02/2025"]),
               format="%d/%m/%Y")
# 3 février 2025, pas 2 mars
```

`errors="coerce"` transforme les valeurs non convertibles en valeurs manquantes : on peut ensuite les compter et décider quoi en faire.

4 Le type category

```
df["categorie"] = df["categorie"].astype("category")
df["categorie"].cat.categories

# mémoire réellement occupée, texte compris
df[["categorie", "region"]].memory_usage(
    deep=True, index=False)
```

- Chaque valeur distincte stockée une seule fois, un code numérique par ligne

Pour les colonnes à **faible cardinalité** : catégorie, région, statut
À convertir **après** le nettoyage du texte

Colonne `categorie` des ventes nettoyées

1,1 Ko

en texte, type `str`

0,1 Ko

en `category`

L'écart devient considérable sur des millions de lignes.

4 Ajouter une valeur à une category

Une colonne `category` n'accepte que des valeurs qui font partie de ses catégories.

Refusé

```
df["region"].fillna("Inconnue")
# TypeError: Cannot setitem on a Categorical
# with a new category (Inconnue), set the
# categories first
```

Avec `region` en `category`, `Inconnue` ne fait pas partie des catégories.

Déclarer d'abord la nouvelle valeur

```
df["region"] = (
    df["region"]
    .cat.add_categories("Inconnue")
    .fillna("Inconnue")
)
```

`cat.add_categories` ajoute la valeur aux catégories, puis `fillna` peut l'écrire.

4 Pratique : TP2, partie 4

Gérer les types, sur de petites Series

Questions	Ce qu'on pratique
4.1	convertir des prix saisis en texte avec `to_numeric` et `errors="coerce"`
4.2	garder des entiers malgré une valeur manquante : le type `Int64`
4.3	convertir des dates au format français, puis utiliser l'accesseur `.dt`

05

Valeurs manquantes, aberrantes et doublons

Détecter | Choisir une stratégie | Dédoublonner

5 Détecter les valeurs manquantes

```
df.isna().sum()                 # nombre par colonne
df.isna().mean().round(3)       # proportion

df[df["region"].isna()]         # lignes concernées
df[df.isna().any(axis=1)]       # au moins un manque
```

Notation	Pour
`NaN`	nombres et texte
`NaT`	dates
`<NA>`	types nullables comme `Int64`

`isna` les détecte toutes.

5 Choisir une stratégie

Stratégie	Code	Quand
Supprimer les lignes	`df.dropna(subset=["region"])`	peu de lignes, colonne indispensable
Supprimer la colonne	`df.drop(columns=["col"])`	colonne très incomplète et peu utile
Valeur fixe	`df["region"].fillna("Inconnue")`	catégorie : l'absence est une information
Statistique	`df["prix"].fillna(df["prix"].median())`	nombre, peu de manques
Propagation	`df["stock"].ffill()`	la dernière valeur connue reste valable
Interpolation	`df["temperature"].interpolate()`	série temporelle qui évolue progressivement

Pas de bonne stratégie universelle : on choisit selon l'usage des données, on justifie, et on note le nombre de lignes concernées.

5 Deux pièges à éviter

Moyenne ou médiane ?

406,80

prix unitaire moyen

215,39

prix unitaire médian

La moyenne est tirée vers le haut par un seul prix à 12 999. La médiane, peu sensible aux extrêmes, est souvent préférable pour remplir.

dropna sans subset

```
df.dropna()
# supprime toute ligne ayant un manque,
# dans n'importe quelle colonne

df.dropna(subset=["region"])
# seulement si region est manquante
```

5 Les valeurs aberrantes

```
anomalies = df[(df["quantite"] <= 0)
               | (df["Prix unitaire"] <= 0)]

# rendre manquant
df["quantite"] = df["quantite"].mask(
    df["quantite"] <= 0
)

# signaler
df["prix_suspect"] = df["Prix unitaire"] > 5000
```

Option	Quand
Supprimer	peu nombreuses et manifestement erronées
Rendre manquantes	puis appliquer une stratégie de remplissage
Signaler	quand on ne peut pas trancher : 480 unités peut être une vraie commande

`mask` remplace par `NaN` là où la condition est vraie. Son inverse : `where`.

5 Les doublons

```
df.duplicated().sum()            # 2
df[df.duplicated(keep=False)]    # toutes les occurrences
df = df.drop_duplicates()

df.duplicated(subset=["id_commande"]).sum()
df["id_commande"].is_unique

# même identifiant : garder la dernière ligne
df.drop_duplicates(subset=["id_commande"],
                   keep="last")
```

- Par défaut : toutes les colonnes comparées, la première occurrence conservée

`subset` : vérifier qu'une **clé** est unique
Même identifiant, lignes différentes : une commande corrigée ? Choisir laquelle garder, par exemple la plus récente avec `keep="last"` après un tri

5 Pratique : TP2, partie 5

Des relevés de magasin avec des trous, un capteur en panne, un fichier clients avec des doublons

Questions	Ce qu'on pratique
5.1	compter les valeurs manquantes et leur proportion par colonne
5.2 et 5.3	interpolation, propagation, valeur fixe ; `dropna` avec ou sans `subset`
5.4	rendre manquantes des mesures impossibles avec `mask`, remplir avec la médiane
5.5	doublons exacts, doublons d'identifiant, garder la dernière ligne

06

Trier, renommer, gérer l'index

sort_values | rename | set_index et reset_index

6 Trier

```
df.sort_values("quantite", ascending=False)

df.sort_values(["categorie", "quantite"],
               ascending=[True, False])

df.sort_index()

df.nlargest(5, "quantite")
```

- Plusieurs colonnes : une liste de colonnes, une liste de sens

`sort_index` rétablit l'ordre des labels
`nlargest` et `nsmallest` : les extrêmes en une ligne

6 Harmoniser les noms de colonnes

```
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
    .str.normalize("NFKD")
    .str.encode("ascii", errors="ignore")
    .str.decode("ascii")
)
```

Avant	Après
`Prix unitaire`	`prix_unitaire`
`Client fidèle`	`client_fidele`
`quantite`	`quantite`

Le *snake_case* : minuscules, sans accents ni espaces, mots séparés par des tirets bas. Les noms deviennent utilisables partout, y compris dans `query`.

Ou colonne par colonne

```
df.rename(columns={"Prix unitaire": "prix_unitaire"})
```

Ce traitement resservira pour chaque nouveau fichier : on l'écrit une fois pour toutes dans une fonction, qui reçoit un DataFrame et en renvoie une copie aux colonnes harmonisées.

6 set_index, reset_index, reindex

```
df["id_commande"].is_unique       # à vérifier avant

df = df.set_index("id_commande")
df.loc[1042]                      # accès direct

df = df.reset_index()

df = df.drop_duplicates().reset_index(drop=True)

df.reindex(columns=["date", "produit", "quantite"])
```

`set_index` : une colonne devient l'index
`reset_index` : l'index redevient une colonne
`drop=True` : renuméroter sans garder l'ancien index, après un filtre ou un dédoublonnage
`reindex` : réorganiser selon une liste ; les labels absents sont remplis de `NaN`

6 Pratique : TP2, partie 6

Trier, renommer, gérer l'index

Questions	Ce qu'on pratique
6.1	trier sur deux colonnes, puis `nlargest`
6.2	écrire la fonction `harmoniser_colonnes` et la tester sur deux tableaux
6.3	vérifier l'unicité, puis `set_index`, `loc` et `reset_index`

07

Une démarche de nettoyage

Un ordre | Des choix notés | Le cas pratique

7 Une démarche de nettoyage

1	2	3	4	5	6	7	8	9
Charger	**Noms de colonnes**	**Doublons**	**Texte**	**Types**	**Aberrantes**	**Manquants**	**Vérifier**	**Exporter**
le brut	snake_case	exacts	espaces, casse, variantes	nombres, dates, catégories	règles métier	stratégie	comme au TP1	Parquet

Le texte avant les catégories et les clés, les aberrantes avant les manquants : on évite les catégories en trop, les doublons manqués et les moyennes faussées.

À chaque étape, on note le choix fait et le nombre de lignes concernées. Cette note fait partie du livrable.

7 Pratique : TP2, partie 7, le cas pratique

Le nettoyage complet des ventes, à partir du fichier brut et de la synthèse du TP1

Questions	Étape de la démarche
7.1 et 7.2	harmoniser les noms de colonnes, supprimer les doublons exacts
7.3 et 7.4	corriger les variantes d'écriture, convertir les dates et les catégories
7.5 et 7.6	repérer les valeurs aberrantes, puis supprimer, rendre manquant ou signaler
7.7	traiter les valeurs manquantes, en justifiant chaque choix
7.8 et 7.9	ajouter les colonnes calculées, refaire les vérifications du TP1

Noter au fur et à mesure chaque choix et le nombre de lignes concernées : la note de fin de notebook fait partie du livrable.

08

Exporter en Parquet

Pourquoi plus compact | Pourquoi plus rapide | Mesurer

8 Pourquoi Parquet est plus compact et plus rapide

1

Stockage par colonne

Les valeurs d'une colonne se ressemblent : elles se compressent bien mieux que des lignes qui mélangent tout.

2

Valeurs répétées encodées

Une colonne comme `region` est stockée sous forme d'un dictionnaire et de codes, comme `category`.

3

Binaire et typé

Pas de texte à analyser à la lecture, les types sont décrits dans le fichier.

On peut ne lire que les colonnes utiles

```
pd.read_parquet("ventes_propres.parquet", columns=["date", "montant"])
```

8 Mesurer : attention aux petits fichiers

```
from pathlib import Path

df.to_csv("ventes_propres.csv", index=False)
df.to_parquet("ventes_propres.parquet",
              index=False)

Path("ventes_propres.csv").stat().st_size

# les types sont conservés
pd.read_parquet(
    "ventes_propres.parquet").dtypes
```

Lignes	CSV	Parquet
59	6,0 Ko	9,9 Ko
295 000	29,5 Mo	0,15 Mo

Sur 59 lignes, les métadonnées du Parquet (schéma, statistiques) pèsent plus que les données. Le gain apparaît avec le volume. Le second cas, les mêmes ventes répétées 5 000 fois, est extrême ; sur des données réelles, le fichier est couramment plusieurs fois plus petit.

8 En pratique

1

Fichier brut

conservé tel quel, jamais écrasé

2

Nettoyage

dans le notebook, sur une copie

3

Parquet propre

pour tous les traitements suivants

4

CSV ou Excel

seulement pour partager

8 Pratique : TP2, partie 8

Exporter le jeu de données propre, puis compléter la note des choix

Questions	Ce qu'on pratique
8.1	exporter en CSV et en Parquet, comparer les tailles, relire le Parquet et ses types
8.2	répéter les ventes 5 000 fois, puis comparer à nouveau, en mégaoctets
Note	compléter le tableau des choix de nettoyage : une ligne par décision

9 TP2 : nettoyage d'un jeu de données

Le notebook `exercices/tp02-nettoyage/tp02.ipynb`, complété au fil de l'après-midi

1

S'exercer

Parties 1 à 6 : un exercice court par section du cours, sur de petits tableaux dont on connaît le résultat.

2

Nettoyer les ventes

Partie 7, le cas pratique : noms de colonnes, doublons, texte, types, valeurs aberrantes et manquantes.

3

Exporter et livrer

Partie 8 : le jeu de données propre en CSV et en Parquet, la comparaison des tailles, la note des choix.

Livrable : un jeu de données propre en CSV et en Parquet, avec une comparaison rapide de leur taille, et une courte note des choix de nettoyage.

À retenir

`loc` : labels, fin incluse. `iloc` : positions, fin exclue
Filtres : parenthèses, `&`, `|`, `~`
Modifier : `df.loc[masque, "col"] = valeur`, en une étape
Vectorisé d'abord, `map` ensuite, `apply` en dernier
`np.where` pour deux valeurs, `pd.cut` pour des classes
`.str` pour le texte, un dictionnaire pour les variantes

`errors="coerce"` rend les échecs de conversion visibles
`category` après le nettoyage du texte ; une nouvelle valeur se déclare avec `cat.add_categories`

- Manquants : choisir, justifier, noter
- Aberrante ne veut pas dire manquante : il faut une règle métier
- Vérifier l'unicité d'une clé avant d'en faire un index
- Le brut ne s'écrase jamais, le propre s'écrit en Parquet